

Algorithms and Data Structures

Sujeong Kim



NP Completeness

Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Tractability

- Some problems are *intractable*:
as they grow large, we are unable to solve them in reasonable time
- What constitutes reasonable time?
 - Standard working definition: *polynomial time*
 - On an input of size n the worst-case running time is $O(n^k)$ for some constant k
- Examples of time complexities:
 - $O(1)$, $O(n)$, $O(n \lg n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(n^n)$, $O(n!)$

Tractability

- Polynomial time
 - $O(1)$: Constant time
 - $O(\lg n)$: Logarithmic time
 - $O(n \lg n)$: Linearithmic time (Log-Linear time)
 - $O(n^k)$: polynomial time
 - E.g., $O(n^2)$ (Quadratic time), $O(n^3)$ (Cubic time)
- Not polynomial time
 - $O(2^n)$: Exponential time
 - $O(n^n)$: Exponential time
 - $O(n!)$: Factorial time
 - $O(2^{n^k})$: Super-polynomial time
 - $O(2^{\sqrt{n}})$: Sub-exponential Time

Tractability

- Why do we prefer polynomial time?
 - Efficiency, scalability, feasibility, simplicity
 - Which is worse? n^{10} vs. 2^n
 - if a system executes 10^9 operations per second
 - $n=10$
 - 10^{10} ($= 10^9 \times 10$): 10 seconds
 - 2^{10} ($\approx 1.24 \times 10^3$): 10^{-6} seconds (microsecond)
 - $n=100$
 - 100^{10} ($\approx 10^{20} = 10^9 \times 10^{11}$) : roughly 3,170 years
 - 2^{100} ($\approx 1.27 \times 10^{30} = 1.27 \times 10^9 \times 10^{21}$): roughly 40 trillion years. (age of the universe is approx. 13.8 billion years)

Polynomial-Time Algorithms

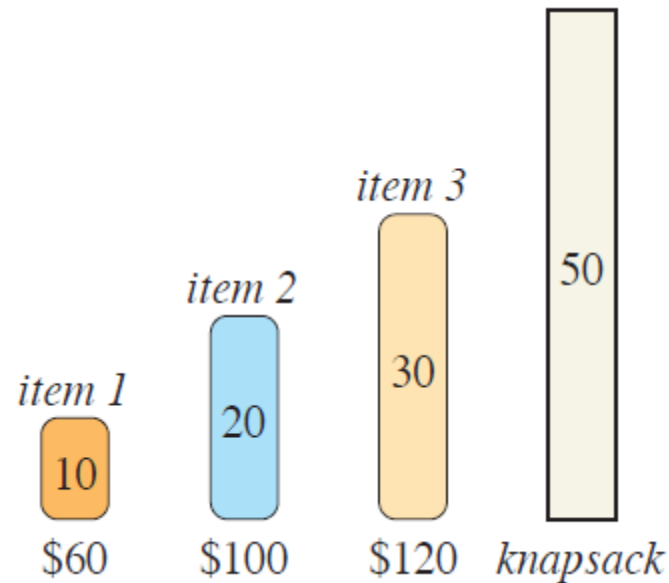
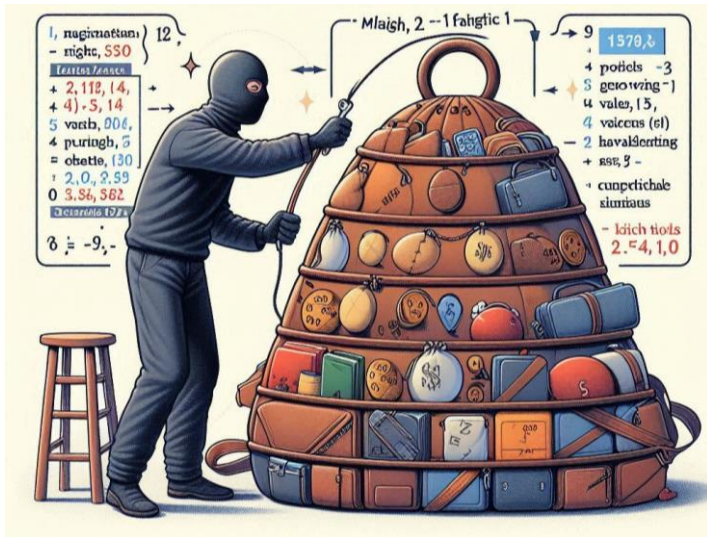
- Are some problems solvable in polynomial time?
 - Of course: many algorithms we've studied (and will study) provide polynomial-time solutions to some problems
- Are all problems solvable in polynomial time?
 - Hard problems (take exponential time)
 - 0/1 Knapsack, Graph coloring, TSP, Hamiltonian Cycle, Sum of Subsets
 - Impossible problems
 - Turing's "Halting Problem" is not solvable by any computer, no matter how much time is given. ([Halting Problem in Theory of Computation - GeeksforGeeks](#))
- Most problems that do not yield polynomial-time algorithms are either optimization or decision problems.

Classic NP-complete/NP-hard problems

Think about brute force approach first

0/1 Knapsack Problem (NP-hard)

- Objective: Maximize the total value of items placed in the knapsack without exceeding the weight capacity.
- **0-1 knapsack**: Binary decision for each item (take it or leave it)



item 1	item 2	item 3	total weight	total value
0	0	0	0	0
0	0	1	30	120
0	1	0	20	100
0	1	1	50	220
1	0	0	10	60
1	0	1	40	180
1	1	0	30	160
1	1	1	60	280

Map Coloring (Graph Coloring) – (NP-complete)

- Determine whether a map can be colored with k colors
- Adjacent regions must have different colors

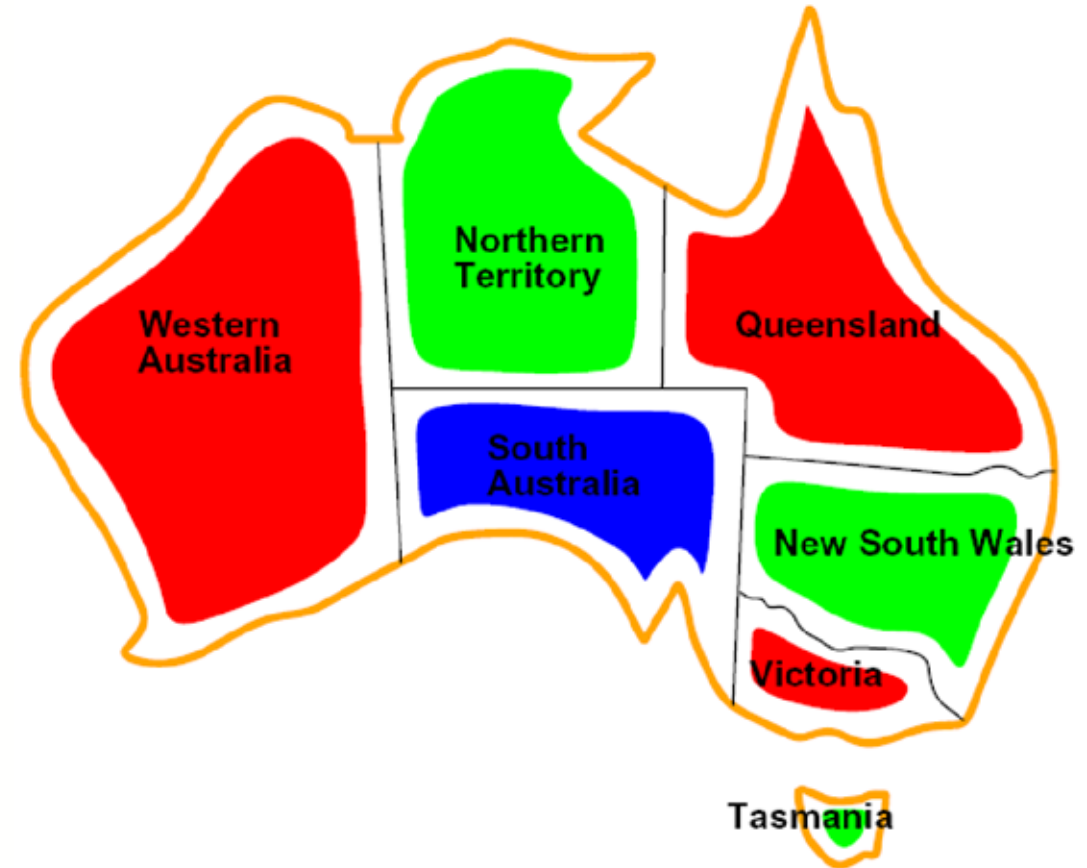
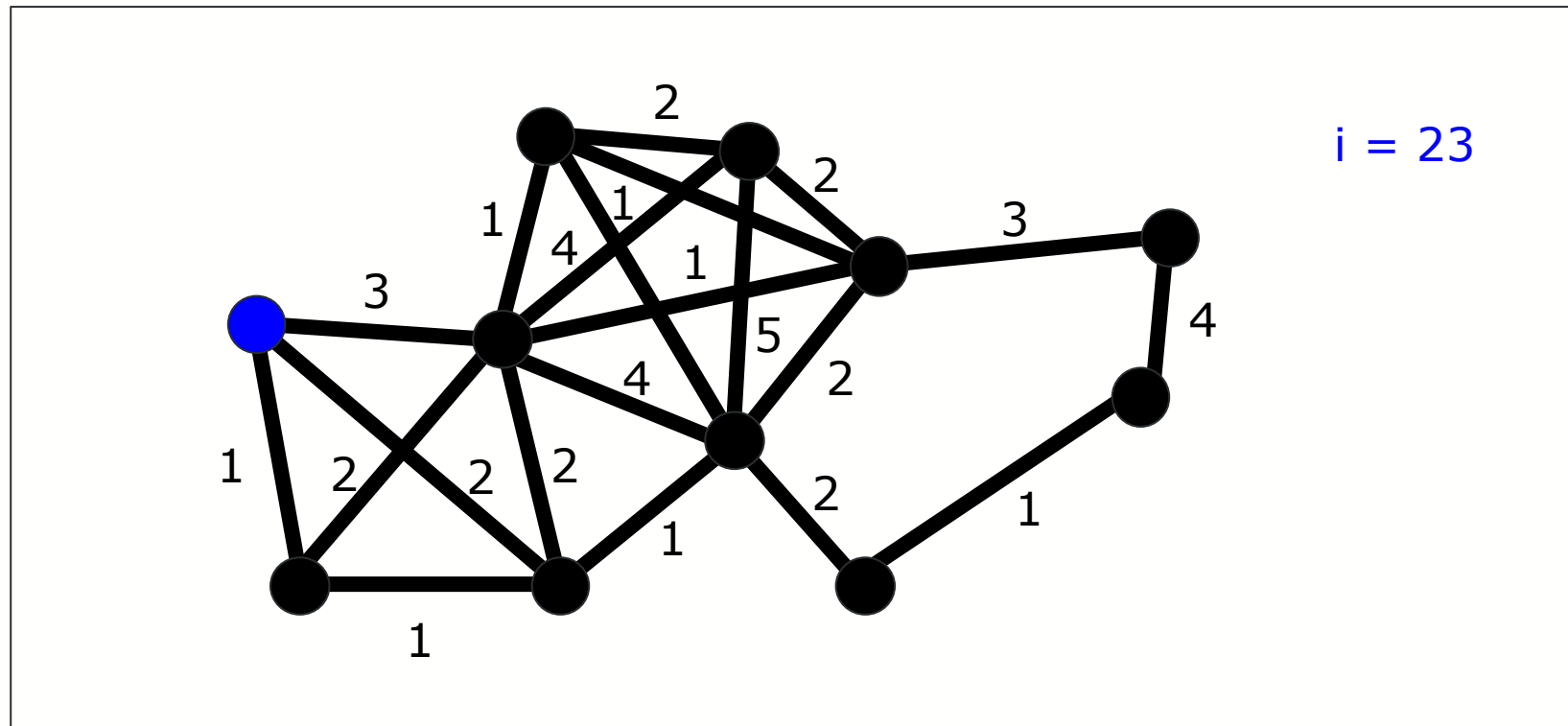


Image: Berkeley CS188

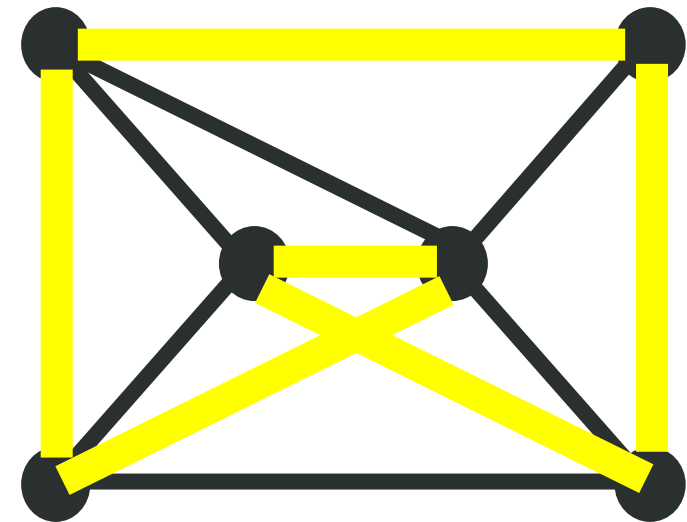
TSP (NP-hard)

- For each two cities, an integer cost is given to travel from one of the two cities to the other. The salesperson wants to make a minimum cost circuit visiting each city exactly once.



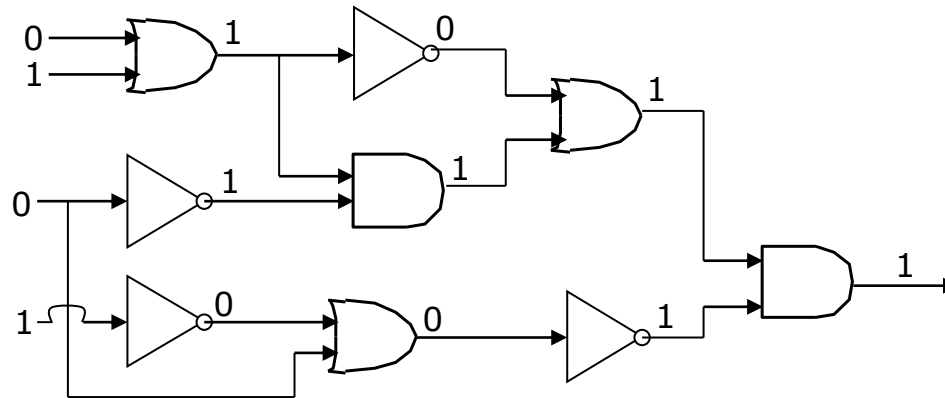
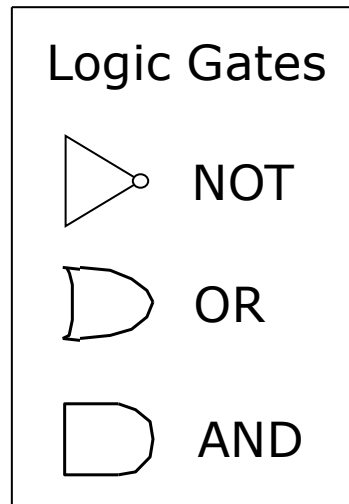
Hamilton Cycle (NP-complete)

- Given a graph $G = (V, E)$, a cycle that visits all the nodes exactly once
- Does not involve optimizing distances or costs
- Decision problem: Does a Hamilton cycle exist?
- Special case of TSP:
 - TSP objective: find the shortest possible route that visits each city (vertex) exactly once and returns to the starting city
 - ❖ $d=1$ adjacent vertices; $d=2$ otherwise



Circuit-SAT (NP-complete)

- Take a Boolean circuit with a single output node and ask whether there is an assignment of values to the circuit's inputs so that the output is “1”



Decision & Optimization Problems

- Decision Problem: computational problem with intended output of “yes” or “no”, 1 or 0
 - Is there a path from node A to node B in a graph?
 - Can a subset of numbers sum up to a given value?
- Optimization Problem: computational problem where we try to find the best feasible solution (e.g., the maximum, minimum, or optimal value) among all valid solutions
 - Finding the shortest path between two nodes in a graph
 - Maximizing profit in a knapsack problem

Decision & Optimization Problems

- Converting optimization into decision
 - Often used to simplify and analyze the problems
- Introduce parameter k and ask if the optimal value for the problem is at most or at least k .
 - Knapsack Problem:
 - Optimization: Maximize the total value of items in the knapsack without exceeding the weight limit.
 - Decision: Is there a subset of items with a total weight $\leq W$ and total value $\geq k$?
 - Traveling Salesman Problem (TSP):
 - Optimization: Find the shortest possible route that visits each city exactly once and returns to the origin.
 - Decision: Is there a route with a total length $\leq k$

Complexity Class P

- Deterministic in nature:
 - These algorithms have predictable behavior and follow a predefined sequence of operations.
 - Given the same input, they always produce the same output.
- We classify problems based on their best-known efficient algorithm.
- If a problem has a polynomial-time solution, it belongs to Class P.
- Example: Matrix Chain Multiplication
 - Naïve approach $\rightarrow$ exponential time ($O(2^n)$)
 - Dynamic programming approach $\rightarrow$ polynomial time ($O(n^3)$)

Complexity Class P

- Solved by conventional computers in polynomial time
 - $O(1)$ Constant Accessing an element in an array
 - $O(\log n)$ Sub-linear Binary search in a sorted array
 - $O(n)$ Linear Traversing an array
 - $O(n \log n)$ Nearly Linear Merge sort
 - $O(n^2)$ Quadratic Bubble sort, Insertion sort
 - $O(n^3)$ Cubic Finding shortest paths between all pairs of nodes in a graph

Sample Problems in P

- Fractional Knapsack
 - Greedy algorithm. $O(n \log n)$
- MST (Minimum Spanning Tree)
 - Kruskal's algorithm $O(E \log E)$
 - Prim's algorithm $O(E \log V)$
- Dijkstra's algorithm
 - $O(V^2)$ or $O(E + V \log V)$
- Longest Common Subsequence (LCS)
 - $O(nm)$ using dynamic programming
- Matrix Chain Multiplication
 - $O(n^3)$ using dynamic programming

Nondeterministic Polynomial-time (NP)

- Definition:

- The class of **decision problems** that can be solved in polynomial time on a nondeterministic machine (or with a nondeterministic algorithm)
- The set of decision problems for which a proposed solution can be verified in polynomial time by a deterministic computer.
- This highlights that the difficulty of NP problems lies in finding the solution, not in verifying its correctness

Deterministic vs. Nondeterministic Machine

- Defined primarily in the context of decision problems (yes/no answers)
- A deterministic machine:
 - One tape, one head, one current state at a time
 - Exactly one possible next move at each step
 - Find the answer by searching
- True non-deterministic machines remain theoretical
 - A nondeterministic computer always “guesses” the right choice at each step
 - Rather than exploring all paths, it magically picks the correct one every time
 - Useful concept model for defining NP – not a real machine

Think-pair-share

- (True/False) Modern computers are still deterministic

Discussion

- Real-world techniques that resemble the concept of non-deterministic methods
 - Quantum computing: explores multiple possibilities simultaneously
 - Parallel computing: distributes work across multiple processes
 - Probabilistic algorithms: make random choices to find good solutions (e.g., genetic algorithms, Monte Carlo methods...)
 - Simulated annealing: escapes local optima by accepting random worse solutions occasionally
- Neural Networks (yes/no)?
 - Is there randomness? When does this happen (e.g., training/inference)?
 - Is it always correct?

P vs. NP

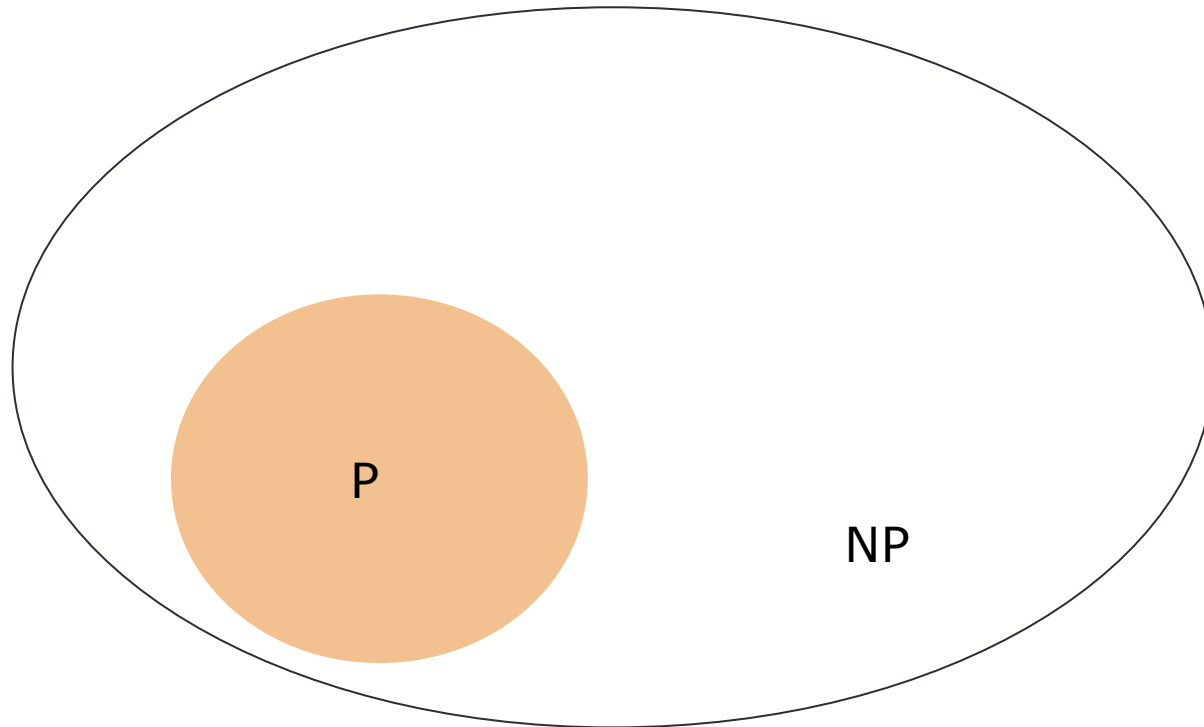
- P = set of problems that can be solved in polynomial time
 - Examples: Fractional Knapsack, ...
- NP = set of problems for which a solution can be verified in polynomial time
 - E.g. Fractional Knapsack,..., Hamiltonian Cycle, CNF SAT, 3-CNF SAT
- Clearly $P \subseteq NP$

Sample Problems in NP but not P

- Traveling Salesman Problem (TSP)
 - Is there a tour shorter than X ?
- Hamiltonian Cycle
 - Is there a path that visits every vertex exactly once?
- Subset Sum Problem
 - Given a set of numbers, is there a subset that sums to a target value?
- Graph Coloring
 - Can a graph be colored using k colors such that no adjacent nodes share a color?
- Satisfiability (SAT)
 - The problem of deciding whether a given Boolean formula is satisfiable

P vs NP

- P: Problems solvable in polynomial time.
- NP: Problems whose solutions can be verified in polynomial time.
- $P \subseteq NP$: Every problem in P is also in NP.
- $P = NP?$: A major unsolved question, with most believing $P \neq NP$.



Open question: $P = NP$?

- Current belief: Most researchers suspect that $P \neq NP$
 - Meaning, there are problems in NP that cannot be solved in polynomial time
- In August 2010, Vinay Deolalikar, a researcher at HP Labs, claimed to have proved that $P \neq NP$, but his proof was found to have flaws

NP-Completeness

- How would you define NP-Complete?
- They are the “hardest” problems in NP

Sudoku (NP-complete)

2			3		8		5	
		3		4	5	9	8	
		8			9	7	3	4
6		7		9				
9	8						1	7
				5		6		9
3	1	9	7			2		
	4	6	5	2		8		
	2		9		3			1

(c) Daily Sudoku Ltd 2006. All rights reserved.

Daily SuDoku: Thu 7-Dec-2006

easy

Sudoku

2			3		8	1	5	
		3		4	5	9	8	
		8			9	7	3	4
6		7		9				
9	8						1	7
		1		5		6		9
3	1	9	7			2		
	4	6	5	2	1	8		
	2		9		3			1

(c) Daily Sudoku Ltd 2006. All rights reserved.

Daily SuDoku: Thu 7-Dec-2006

easy

Sudoku

2			3		8	1	5	
		3		4	5	9	8	2
		8	2		9	7	3	4
6		7		9				
9	8	2					1	7
		1		5		6		9
3	1	9	7			2		
	4	6	5	2	1	8		
	2		9		3			1

(c) Daily Sudoku Ltd 2006. All rights reserved.

Daily SuDoku: Thu 7-Dec-2006

easy

Sudoku

2	9	4	3	7	8	1	5	6
1	7	3	6	4	5	9	8	2
5	6	8	2	1	9	7	3	4
6	5	7	1	9	2	3	4	8
9	8	2	4	3	6	5	1	7
4	3	1	8	5	7	6	2	9
3	1	9	7	8	4	2	6	5
7	4	6	5	2	1	8	9	3
8	2	5	9	6	3	4	7	1

(c) Daily Sudoku Ltd 2006. All rights reserved.

Daily SuDoku: Thu 7-Dec-2006

easy

Sudoku

	F		2						6			C	B	3	
	C				4	8	E	A			0		D		
D	A	8			3		2	7	F			6		5	
6			E	D	F		C		8						7
	9	3		7					A						2
E						6	F	5		8	4		3		1
C	8		1	3	9	D		0	2		E				
	D		6		5	E	B		1					0	4
9	6					1		F	3	2		0		A	
				4		A	8		D	0	9	B		2	5
2		A		0	D		5	6	C						F
5						2					A		4	8	
B						4		1		A	2	F			0
	0		7			F	3	C		D			2	9	B
		5		1			A	9	0	B				D	
	2	D	A			9						1		4	

(c) Daily Sudoku Ltd 2006. All rights reserved.

Sudoku

0	F	9	2	A	7	5	1	4	6	E	D	C	B	3	8
7	C	1	3	6	4	8	E	A	B	5	0	2	D	F	9
D	A	8	4	9	3	B	2	7	F	C	1	6	0	5	E
6	5	B	E	D	F	0	C	2	8	9	3	4	A	1	7
4	9	3	5	7	1	C	0	D	A	F	B	8	E	6	2
E	B	7	0	2	A	6	F	5	9	8	4	D	3	C	1
C	8	F	1	3	9	D	4	0	2	6	E	5	7	B	A
A	D	2	6	8	5	E	B	3	1	7	C	9	F	0	4
9	6	4	8	E	B	1	7	F	3	2	5	0	C	A	D
3	7	C	F	4	6	A	8	E	D	0	9	B	1	2	5
2	1	A	B	0	D	3	5	6	C	4	8	7	9	E	F
5	E	0	D	F	C	2	9	B	7	1	A	3	4	8	6
B	3	6	9	C	E	4	D	1	5	A	2	F	8	7	0
1	0	E	7	5	8	F	3	C	4	D	6	A	2	9	B
8	4	5	C	1	2	7	A	9	0	B	F	E	6	D	3
F	2	D	A	B	0	9	6	8	E	3	7	1	5	4	C

(c) Daily Sudoku Ltd 2006. All rights reserved.

Monster Daily SuDoku: Thu 7-Dec-2006

medium

Sudoku

2			3		8		5	
		3		4	5	9	8	
		8			9	7	3	4
6		7		9				
9	8						1	7
				5		6		9
3	1	9	7			2		
	4	6	5	2		8		
	2		9		3			1

© Daily Sudoku Ltd 2006. All rights reserved. easy

	F	2					6		C	B	3
	C			4	8	E	A		0	D	
D	A	8		3	2	7	F		6		5
6			E	D	F		8				7
	9	3		7			A				2
E					6	F	5		8	4	
C	8	1	3	9	D		0	2	E		
	D	6		5	E	B	1			0	4
9	6				1	F	3	2		0	A
			4		A	8	D	0	9	B	2
2		A		0	D	5	6	C			F
5					2				A		4
B				4		1	A	2	F		0
	0	7			F	3	C	D		2	9
	5		1		A	9	0	B		D	
	2	D	A		9				1		4

© Daily Sudoku Ltd 2006. All rights reserved. medium

⋮

$n \times n \times n$

- Suppose it takes you $S(n)$ to solve $n \times n \times n$ Sudoku puzzle.
- Let $N = n^2$, number of elements in one row, column, subgrid.
- Let $V(N)$ be the time required to verify the solution
 - for each row, column, and subgrid, check that it contains all numbers from 1 to N
 - Brute-force **verification**: $V(N) = O(3 \times N \times N) = O(N^2) = O(n^4)$
- Question: is there some constant c such that

$$S(n) \leq n^c?$$

(Does $S(n)$ grow at most polynomially?)

* The exact complexity of $S(n)$ is not definitively known. It is believed to be exponential or super-polynomial in the worst case. → NP-complete

NP-complete problems

- A decision problem D is NP-complete iff
 - $D \in \text{NP}$
 - Every problem in NP is polynomial-time reducible to D
- NP-complete problems:
 - The hardest problems in NP.
 - They can be used to solve any other NP problems through polynomial-time reduction

NP-complete problems

- CNF-SAT: Given a Boolean formula in conjunctive normal form (CNF), the problem is to decide whether there exists an assignment of truth values to variables that makes the formula true. (e.g., $(A \vee \neg B) \wedge (B \vee C) \wedge (\neg A \vee \neg C)$)
- Cook's theorem (1971): CNF-Satisfiability is NP-complete
 - First problem proven to be NP-complete
 - Established a foundation for identifying other NP-complete problems

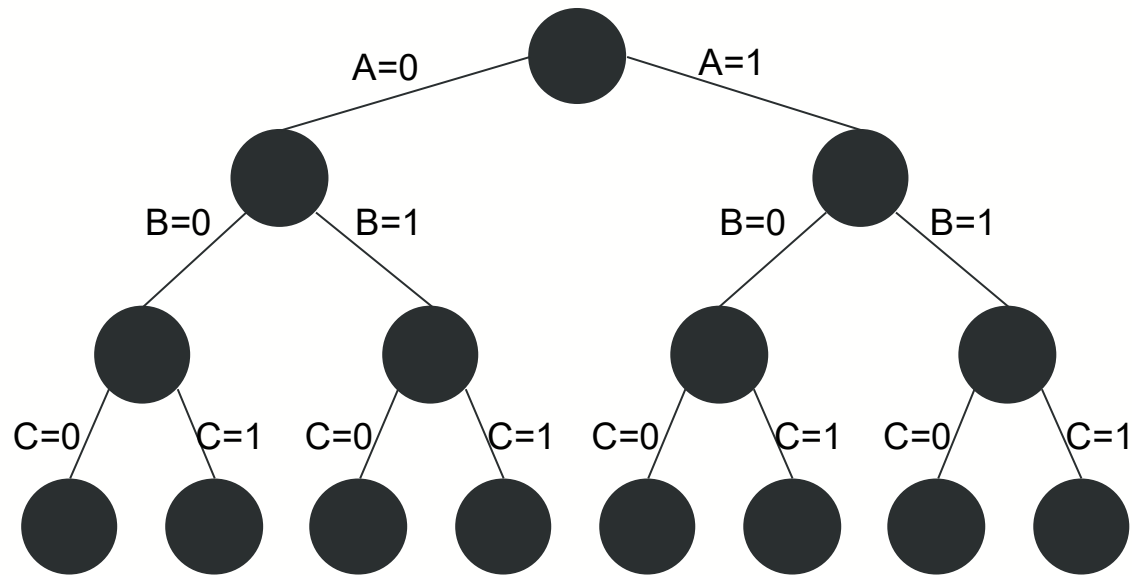
Reduction

- A problem R can be reduced to another problem Q if any instance of R can be rephrased to an instance of Q , the solution to which provides a solution to the instance of R
 - This rephrasing is called a transformation
 - The transformation must be efficient, ideally taking polynomial time.
- Intuitively: If R reduces in polynomial time to Q , R is “no harder to solve” than Q

Reduction Examples

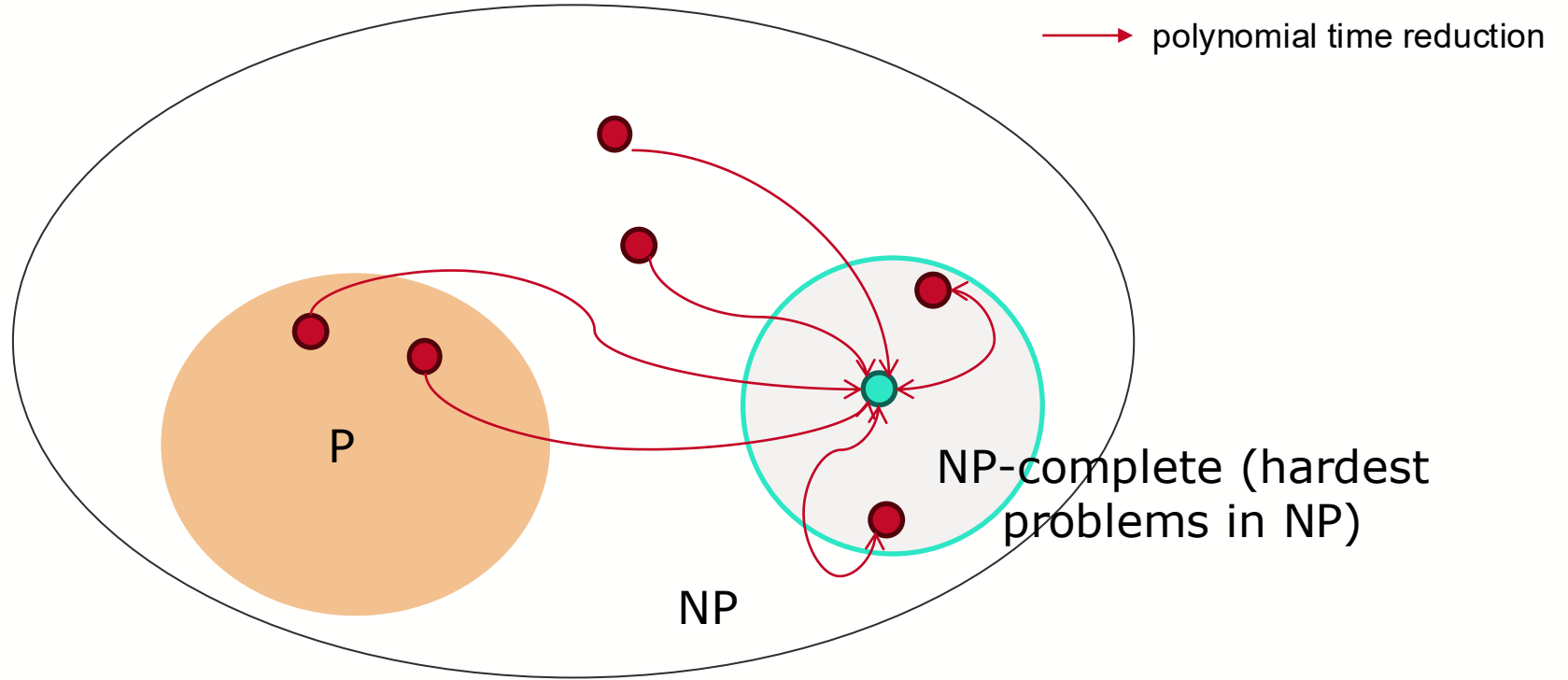
- Use an algorithm for B (already known solution) to solve A (unknown solution)
- Multiplication and addition
 - Problem: Calculate 5×3
 - Reduction: Transform it into $5 + 5 + 5$
 - Multiplication is reduced to addition problem
- Least Common Multiple (LCM) and Greatest Common Divisor (GCD)
 - $\text{lcm}(m, n) = m * n / \text{gcd}(m, n)$
 - $\text{lcm}(m, n)$ problem is reduced to $\text{gcd}(m, n)$ problem

Reduction



- CNF-SAT
 - $(A \vee \neg B) \wedge (B \vee C) \wedge (\neg A \vee \neg C)$
- 0/1 Knapsack
 - Item={A,B,C}
 - Profit = {10,8,12}
 - Weight={5, 4, 3}
 - Num. Items=3, Knapsack capacity=8
- Each variable in CNF-SAT (A,B,C) corresponds to an item in knapsack
- Selecting an item represents setting the variable to true, and not selecting represents false.
- Each clause in the CNF translate into a knapsack constraint (e.g., selecting or excluding an item)

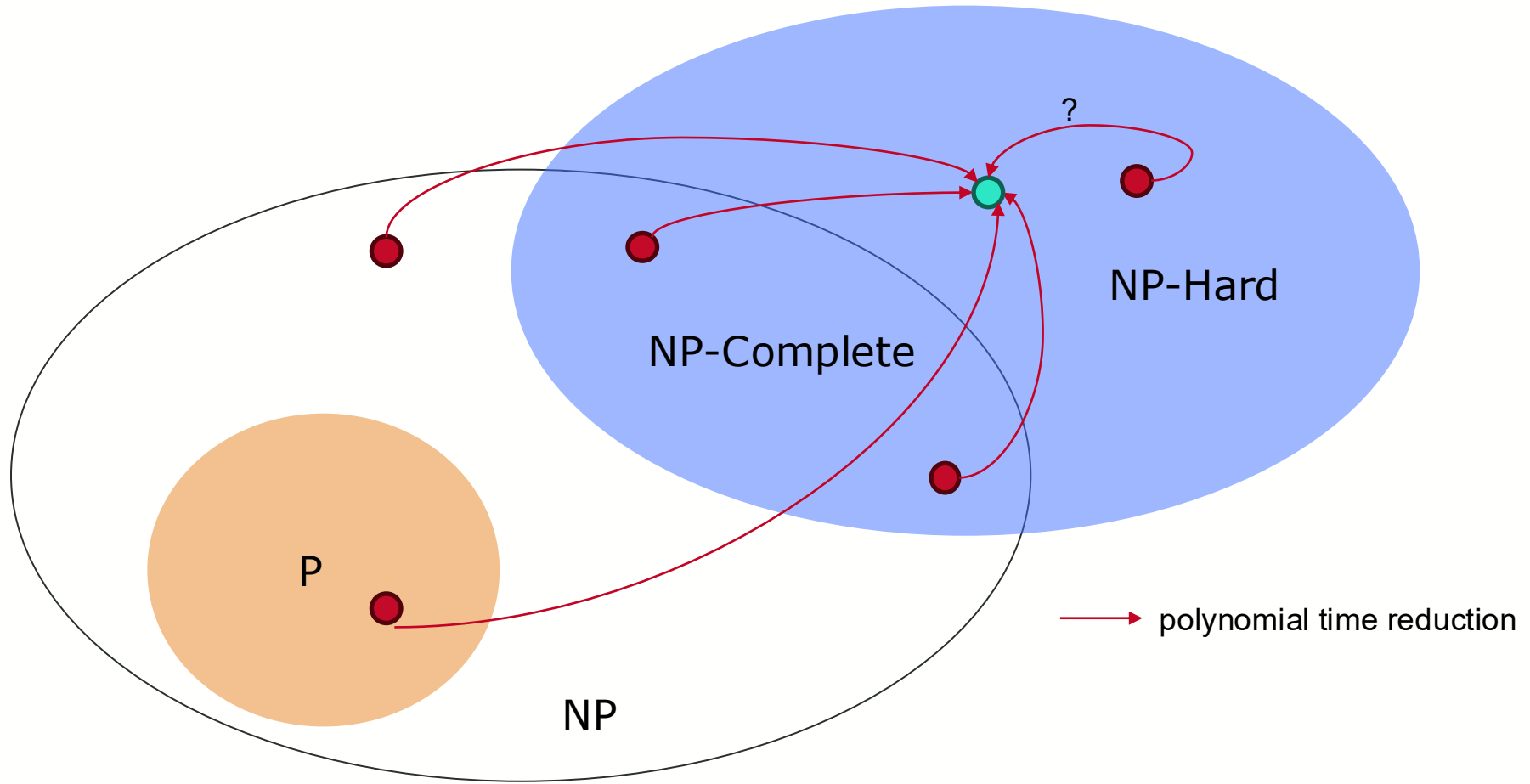
NP-complete



NP-Hard and NP-Complete

- A problem M is NP-hard if every problem L in NP can be transformed (or reduced) to M using a polynomial-time reduction
 - M does not need to be in NP (It might not even be a decision problem)
 - If you can solve M efficiently, you can solve any problem in NP efficiently by converting into M
- For every L that is a member of NP, $L \text{ poly} \rightarrow M$
- NP-complete problems are both NP-hard and in NP (by definition)
- If problem M is NP-hard and also in the class of NP itself, then M is NP-complete

NP-Hard



NP-Hard and NP-Complete

- Traveling Salesman Problem (TSP)
 - Objective: find the shortest possible route that visits each city exactly once and returns to the origin city.
 - TSP Decision Problem (NP-Complete)
 - Is there a route with a total length less than or equal to a given value k ?
 - **NP** because Given a route, we can verify if its total length is less than or equal to k in polynomial time
 - **NP-Hard** because Every problem in NP can be reduced to the TSP decision problem in polynomial time
 - TSP Optimization Problem (NP-Hard)
 - Find the shortest route that visits each city exactly once and returns to the origin.

NP-Hard

- NP-hard problems encompass a wide range of challenges, many of which are not in NP due to their complexity and the inability to verify solutions in polynomial time.

(Think-Pair-Share) $P=NP$?

- What does it imply?
 - Every decision problem for which a solution can be verified in polynomial time can also be solved in polynomial time
 - What about NP-hard problems?
- $P=NP$, should I care?

P=NP? Implications in the Real-World

- Many encryption schemes rely on problems that are believed to be hard to solve but easy to verify. If $P=NP$, existing cryptographic methods will become insecure.
- Complex scheduling, logistics, and resource allocation problems that currently take enormous computation to solve would suddenly become efficiently solvable.
- Many difficult problems in AI, such as learning optimal strategies or reasoning, would become easier to compute.
- Mathematical discoveries could explode in complexity and depth, since finding proofs will be as easy as verifying them.

(Think-Pair-Share) Follow-up Questions

- Does Artificial Super-intelligence mean $P=NP$?
- If someone proved $P=NP$ tomorrow, but the algorithm was theoretically polynomial yet practically unusable (e.g., $O(n^{1000000})$). Would it still matter?
 - Theoretically yes, practically no
 - Most problems would still be computationally infeasible for any real input
 - Look up “Galactic algorithms”

NP-Hard and NP-Complete

- Proving NP-Hardness
 - Restriction
 - Local replacement
 - Component design

NP-Hard and NP-Complete

- **Restriction:** Show that a known NP-complete problem M is a special case of L
 - If M is a special case of L , then L is at least as hard as M
 - L must solve all the instances of M , along with possibly more general cases.
 - Example:
 - $M=3\text{-SAT}$ problem (a special case of SAT where each clause has exactly three literals).
 - $L=\text{general SAT problem}$
 - Then L includes all instances of 3-SAT, making SAT at least as hard as 3-SAT.

NP-Hard and NP-Complete

- **Local replacement:** reduce a known NP-complete problem M to L by dividing instances of M and L into “basic units” then showing each unit of M can be converted to a unit of L
 - Break down the problem instances into smaller, manageable pieces. Then, demonstrate that each small piece of M can be transformed into a corresponding piece of L effectively.
 - Example: reduce the Hamiltonian Cycle problem to another graph problem by converting each vertex and edge in the Hamiltonian Cycle instance into equivalent structures in L . We show that solving L solves the original problem M .

NP-Hard and NP-Complete

- **Component design:** reduce a known NP-complete problem M to L by building components for an instance of L that enforce important structural functions for instances of M .
 - Construct parts (components) for L that replicate the critical features and constraints of M . By designing these components, we ensure that solving L will inherently solve the key aspects of M .
 - Example: Reducing the SAT problem to a circuit satisfiability problem. Design logic gate components in the circuit that correspond to the logical operations in the SAT formula.

Approaches for NP-Complete/NP-Hard Problems

- Exact Algorithms
 - Branch and Bound, Backtracking, Dynamic Programming
- Approximation Algorithms:
 - Deterministic: guaranteed/provable bounds on solution quality
 - PTAS (Polynomial-Time Approximation Scheme), Christofides algorithm (1.5x for TSP)
 - Some greedy algorithms with proven approximation ratios
- Heuristic Algorithms: no formal guarantee
 - Most greedy algorithms in practice
 - Simulated Annealing, Genetic Algorithms (also probabilistic)
- Probabilistic Algorithms
 - Monte Carlo Methods (random sampling)

Transforming Intractable Problems with Machine Learning & AI

- Probabilistic & heuristic approach
- Offer practical, near-optimal solutions that are computationally feasible for real-world applications
- Examples:
 - Traveling Salesman Problem (TSP): NNs learn heuristics for near-optimal routes
 - Knapsack problem: Reinforcement learning
 - Max-CSPs (e.g., Sudoku): Graph Neural Networks (GNNs) + semidefinite programming (SDP)
 - Game Playing: Deep neural networks achieves superhuman performance in complex games
- Pros and cons of using machine learning techniques
 - Pros: speed and efficiency, scalability, adaptability
 - Cons: data dependency, complexity, approximation

Computational Complexity of Various NP Problem Solving Approaches

Approach	Complexity	Guarantee of Optimality	Practical Feasibility
Brute Force	Exponential	Yes	No (Too slow)
Branch and bound, Backtracking	Exponential (Worst case)	Yes	Limited (Depends on pruning efficiency)
Dynamic Programming	Pseudo-Polynomial*	Yes	Limited (High memory)
Approximation	Polynomial	Near-optimal	Yes (Fast)
Heuristic	Polynomial	No	Yes (Efficient)
Probabilistic	Polynomial	No	Yes (Good for large instances)
ML-Based	Variable	No	Yes (Adaptive)

* In **pseudo-polynomial time**, complexity depends on the **actual numerical value** of the input, which can make a problem appear tractable for small numbers but infeasible for large values.

Heuristic, probabilistic, and ML-based methods **do not guarantee** that their solutions are consistently close to optimal.

Complexity Classes

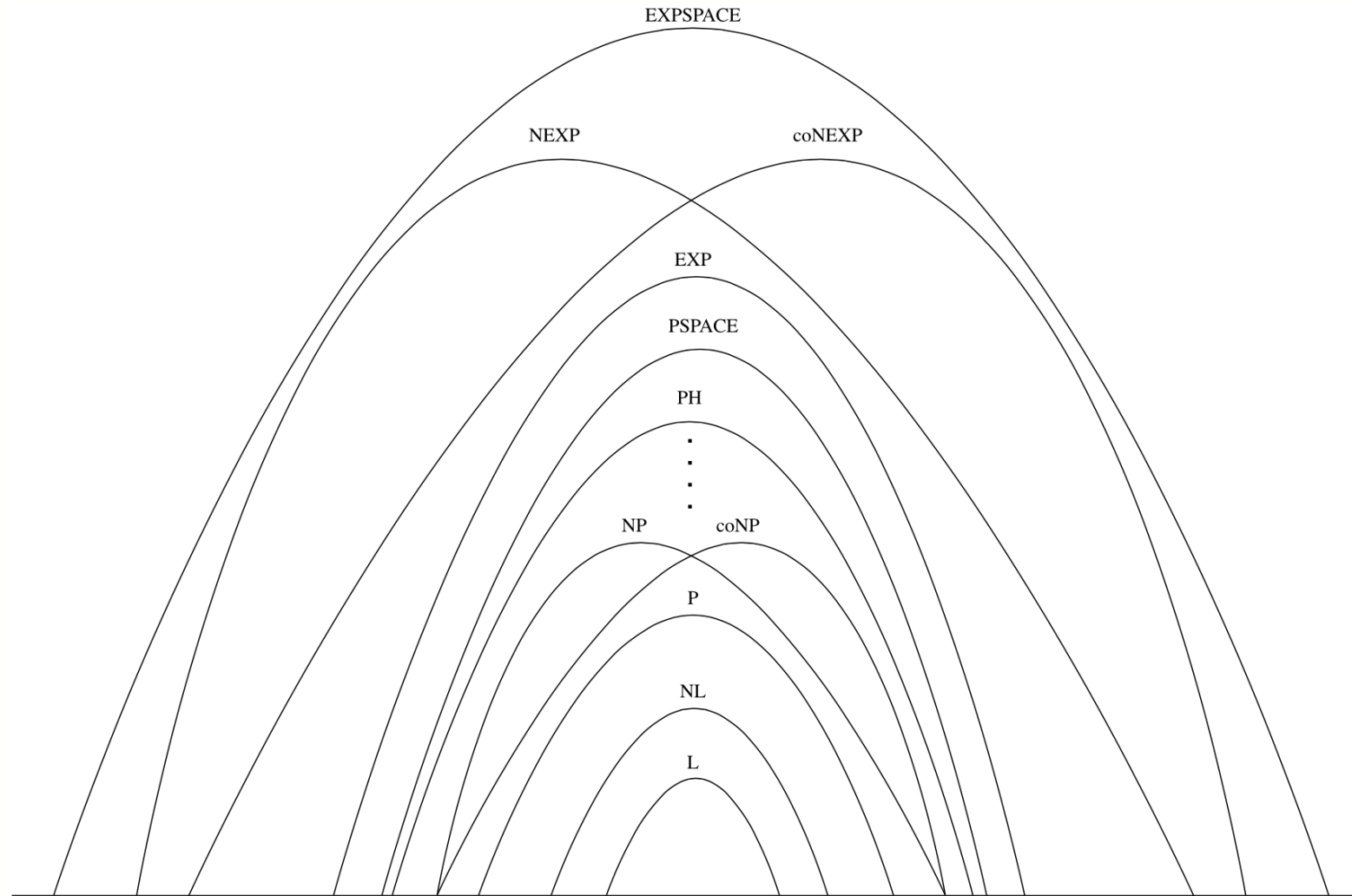


Figure 2. Inclusion relationships among major complexity classes. The only depicted inclusions which are currently known to be proper are $L \subsetneq PSPACE$ and $P \subsetneq EXP$.